

LAB ACTION PLAN FOR WEEK 6

CSE-B

24BD1A0539-P.BHARGAVI

Email-bhargavipabbathi17@gmail.com

Objectives:

Students will able to:

- Learn how to define and run multiple interdependent services (e.g., web server, database) in a single configuration file.
- Gain skills in writing and interpreting docker-compose.yml files for service setup.
- Deploy the same setup across different machines without manual configuration.
- Configure container networking and persistent storage within Compose.
- Reduce setup time and enable faster iteration during application development.

Students will perform Hands-on running multi container applications using docker compose and make document that include the execution of each step along with scenario-based question solutions after completing the tasks.

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Try the new cross-platform PowerShell https://aka.ms/pscore6

PS C:\Users\fs> docker --version
Docker version 29.0.1, build eedd969
PS C:\Users\fs> docker compose version
Docker Compose version v2.40.3-desktop.1
PS C:\Users\fs> █
```

What is Docker Compose?

Docker Compose is a tool used to define and run multi-container Docker applications. It allows you to define services, networks, and volumes that your application needs, all in a single file. This makes it easier to manage complex applications that require multiple containers (e.g., a web server and a database).

Structure of a Docker Compose File

A Docker Compose file is written in YAML (YAML Ain't Markup Language), a human-readable data format used for configuration. In the Docker context, it helps define the services, networks, and volumes needed for an application.

Basic Components of a Docker Compose YAML File

1. **Version:** Specifies which version of the Docker Compose file format you're using. Each version comes with its own set of features.

```
version: '3.8'
```

2. **Services:** This section defines all the containers you want to run for your application. Each service represents a container (like a web server or database).

Each service includes:

- **Image:** The Docker image to use.
- **Ports:** Ports to map between the container and your host machine.
- **Environment:** Environment variables required by the service.
- **Dependencies:** Which other services a container depends on.

Example:

```
services:
  web:
    image: nginx
    ports:
      - "8060:80"
  db:
    image: tomeet
    ports:
      - "8050:8080"
```

3. **Networks:** Define networks to allow different services to communicate with each other. Docker Compose automatically creates a default network if not specified.
4. **Volumes:** Used for persistent storage, allowing data to persist even after the container stops or is removed.

Example Docker Compose File (Simple)

Here's a basic example of a Docker Compose file that runs WordPress and MySQL together:

```
version: '3.8' # Docker Compose file format version

services:
  wordpress: # WordPress service
    image: wordpress:latest
    ports:
      - "8081:80" # Map port 80 of the container to port 8080 of the host
    environment:
      WORDPRESS_DB_HOST: db:3306 # Database host
      WORDPRESS_DB_USER: wordpress
      WORDPRESS_DB_PASSWORD: wordpress
      WORDPRESS_DB_NAME: wordpress
    depends_on:
      - db # Ensures the db service starts first

  db: # MySQL service
    image: mysql:5.7
    environment:
      MYSQL_ROOT_PASSWORD: rootpassword
      MYSQL_DATABASE: wordpress
      MYSQL_USER: wordpress
      MYSQL_PASSWORD: wordpress
```

How to Run It

To run a multi-container setup like the one above:

- 1. Save the file as docker-compose.yml.**

Or

```
docker-compose.yaml
```

- 2. To Start the compose**

```
docker-compose up -d
```

- 3. To stop the containers**

```
docker-compose down
```

- 4. To scale the container**

```
docker-compose up --scale <service name>=2 -d
```

1. Define and run multiple interdependent services

Task:

I. Create a new folder compose-lab

Inside it, create a file docker-compose.yml with the following content:

```
version: "3.9"
```

```
services:
```

```
  web:
```

```
    image: nginx:latest
```

```
    ports:
```

```
      - "8080:80"
```

```
  db:
```

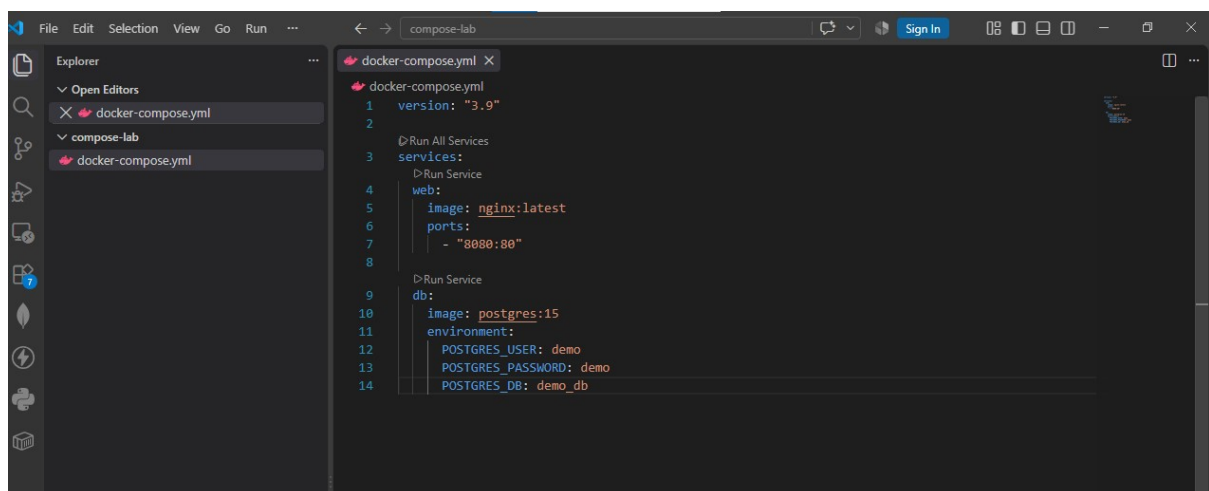
```
    image: postgres:15
```

```
    environment:
```

```
      POSTGRES_USER: demo
```

```
      POSTGRES_PASSWORD: demo
```

```
      POSTGRES_DB: demo_db
```

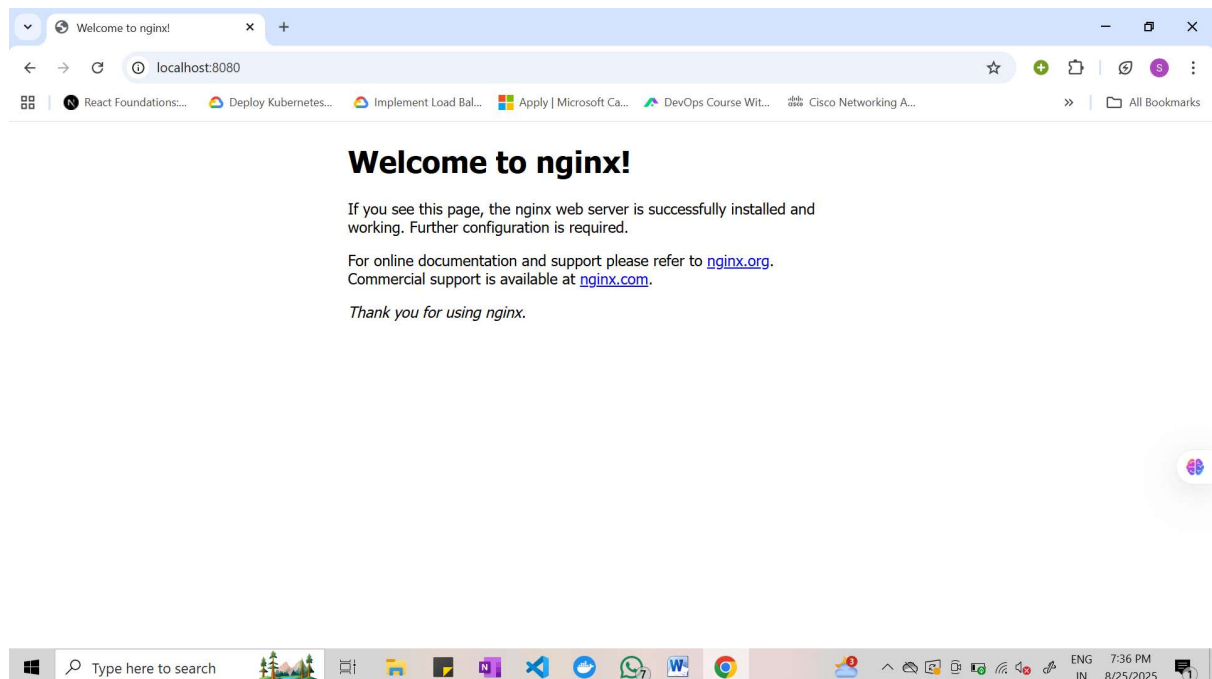


II. Run the setup:

docker compose up -d

```
PS C:\Users\fs\Downloads\compose-lab> docker compose up -d
time="2026-09-18T10:50:10+05:30" level=warning msg="C:\\Users\\fs\\Downloads\\compose-lab\\docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion"
Container compose-lab-db-1 Running
Container compose-lab-web-1 Started
PS C:\Users\fs\Downloads\compose-lab>
```

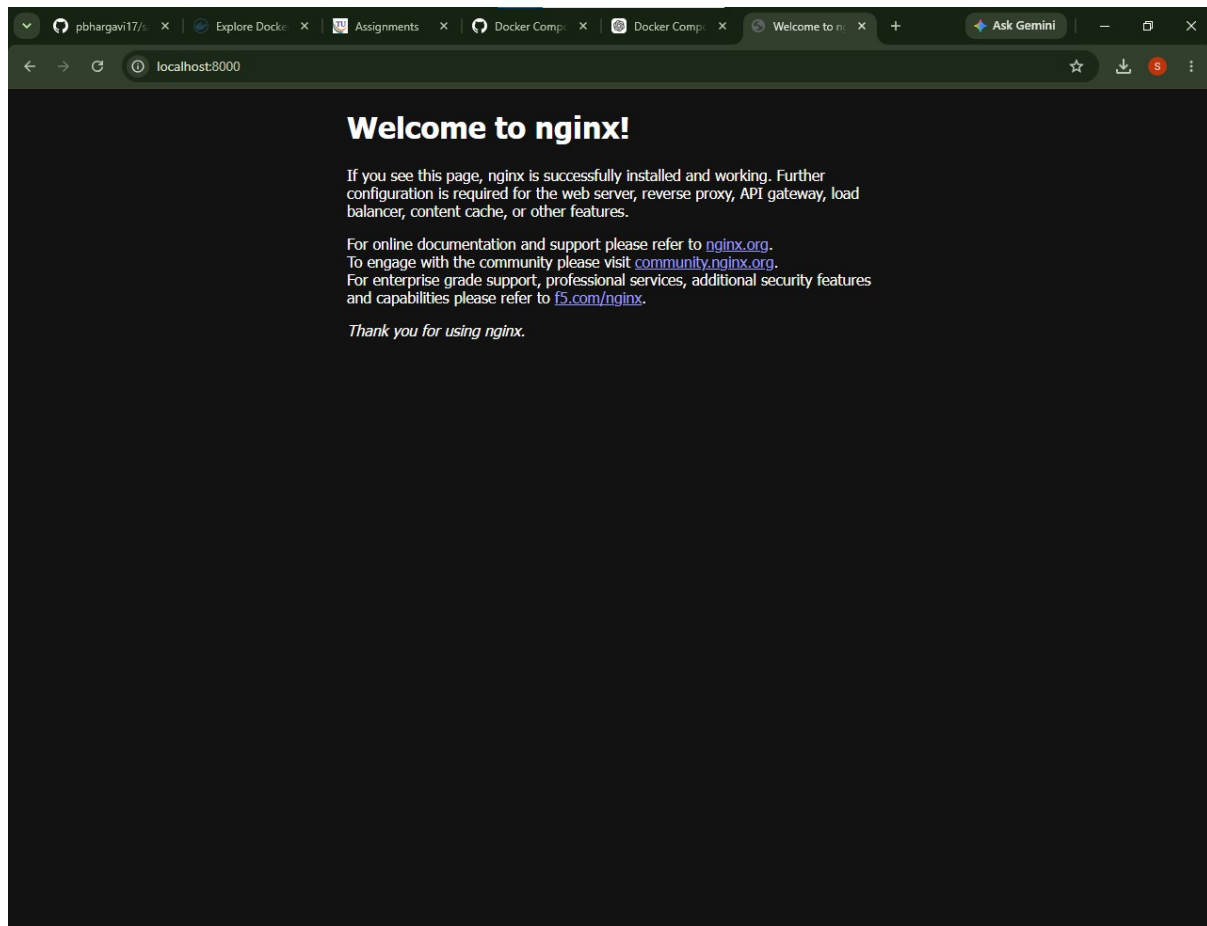
III. Open your browser and visit: <http://localhost:8080>.



IV. Expected Output:

Nginx welcome page is displayed.

db container runs in the background.



```
PS C:\Users\fs\Downloads\compose-lab> docker compose ps
time="2026-09-18T10:51:54+05:30" level=warning msg="C:\\Users\\fs\\Downloads\\compose-lab\\docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion"
NAME                IMAGE              COMMAND                SERVICE  CREATED      STATUS        PORTS
compose-lab-db-1    postgres:15       "docker-entrypoint.s..." db        2 minutes ago Up 2 minutes  5432/tcp
compose-lab-web-1   nginx:latest       "/docker-entrypoint..." web       About a minute ago Up About a minute 0.0.0.0:8000->80/tcp, [::]:8000->80/tcp
PS C:\Users\fs\Downloads\compose-lab>
```

2. Write and interpret docker-compose.yml files

Task:

I. Modify docker-compose.yml to add a Redis cache:

redis:

image: redis:alpine

II. Add a depends_on so web waits for Redis:

web:

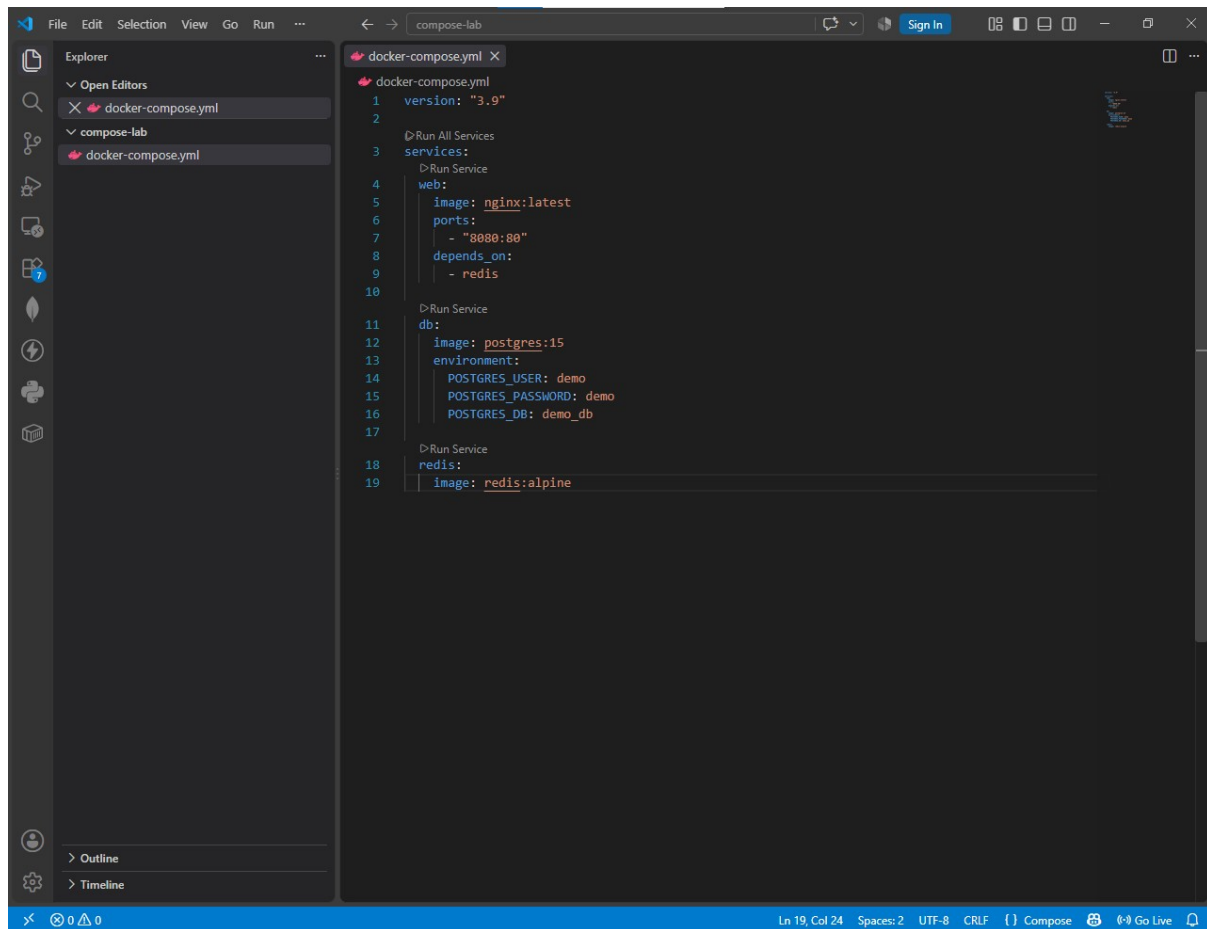
image: nginx:latest

ports:

- "8080:80"

depends_on:

- redis



III. Restart the setup:

docker compose up -d

docker compose ps

```

network address/port is not fully permitted.
PS C:\Users\fs\Downloads\compose-lab> docker compose up -d
time="2026-09-18T10:56:23+05:30" level=warning msg="C:\\Users\\fs\\Downloads\\compose-lab\\docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion"
Container compose-lab-redis-1 Running 0.0s
Container compose-lab-web-1 Started 1.5s
Container compose-lab-db-1 Running 0.0s
PS C:\Users\fs\Downloads\compose-lab> docker compose ps
time="2026-09-18T10:56:55+05:30" level=warning msg="C:\\Users\\fs\\Downloads\\compose-lab\\docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion"
NAME                IMAGE                COMMAND                SERVICE    CREATED          STATUS              PORTS
compose-lab-db-1    postgres:15         "docker-entrypoint.s..." db         7 minutes ago    Up 7 minutes        5432/tcp
compose-lab-redis-1 redis:alpine         "docker-entrypoint.s..." redis     49 seconds ago   Up 48 seconds       6379/tcp
compose-lab-web-1   nginx:latest        "/docker-entrypoint..." web       31 seconds ago   Up 30 seconds       0.0.0.0:8000->80/tcp, [::]:8000->80/tcp
PS C:\Users\fs\Downloads\compose-lab>

```

IV. Expected Output:

Three services (web, db, redis) are listed as running.

3. Deploy across different machines

Task:

I. Zip your compose-lab folder.

Transfer it to another machine with Docker Compose installed.

```

PS C:\Users\fs\Downloads\compose-lab> cd ..
PS C:\Users\fs\Downloads> Compress-Archive -Path .\compose-lab -DestinationPath .\compose-lab.zip
PS C:\Users\fs\Downloads> dir

Directory: C:\Users\fs\Downloads

Mode                LastWriteTime         Length Name
----                -
d-----          18-09-2026   10:47             compose-lab
d-----          16-09-2026   12:09             docker-compose
d-----          18-09-2026   10:33             SE-DOCKER
-a-----          18-09-2026   10:26      856655 24BD1A0539_DOCKER-CLI.docx
-a-----          10-09-2026   09:42      18665  CN_LAB_SET-1_10-09-26.docx
-a-----          18-09-2026   10:57        329  compose-lab.zip

```

II. Run:

docker compose up -d

```

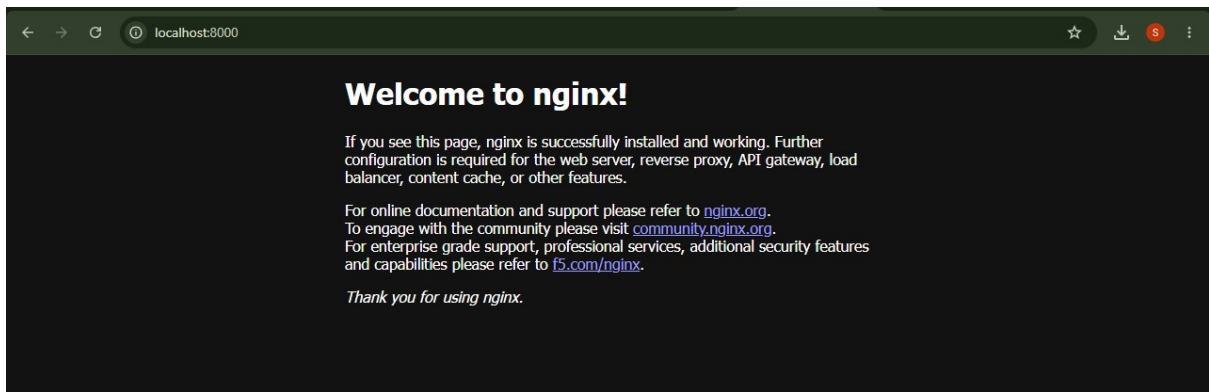
PS C:\Users\fs\Downloads\compose-lab> docker compose up -d
time="2026-09-18T11:04:58+05:30" level=warning msg="C:\\Users\\fs\\Downloads\\compose-lab\\docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion"
[+] Running 1/3
Container compose-lab-redis-1 Started 0.4s
Container compose-lab-db-1 Started 0.4s
Container compose-lab-web-1 Started 0.4s
PS C:\Users\fs\Downloads\compose-lab>

```

Check that Nginx and Postgres work there as well.

III. Expected Output:

The same services run on the new machine without changes.



4. Networking and persistent storage

Task:

- I. **Update your docker-compose.yml to add a custom network and volume:**

networks:

app-net:

volumes:

db-data:

services:

web:

image: nginx:latest

ports:

- "8080:80"

networks:

- app-net

depends_on:

- db

db:

image: postgres:15

environment:

POSTGRES_USER: demo

POSTGRES_PASSWORD: demo

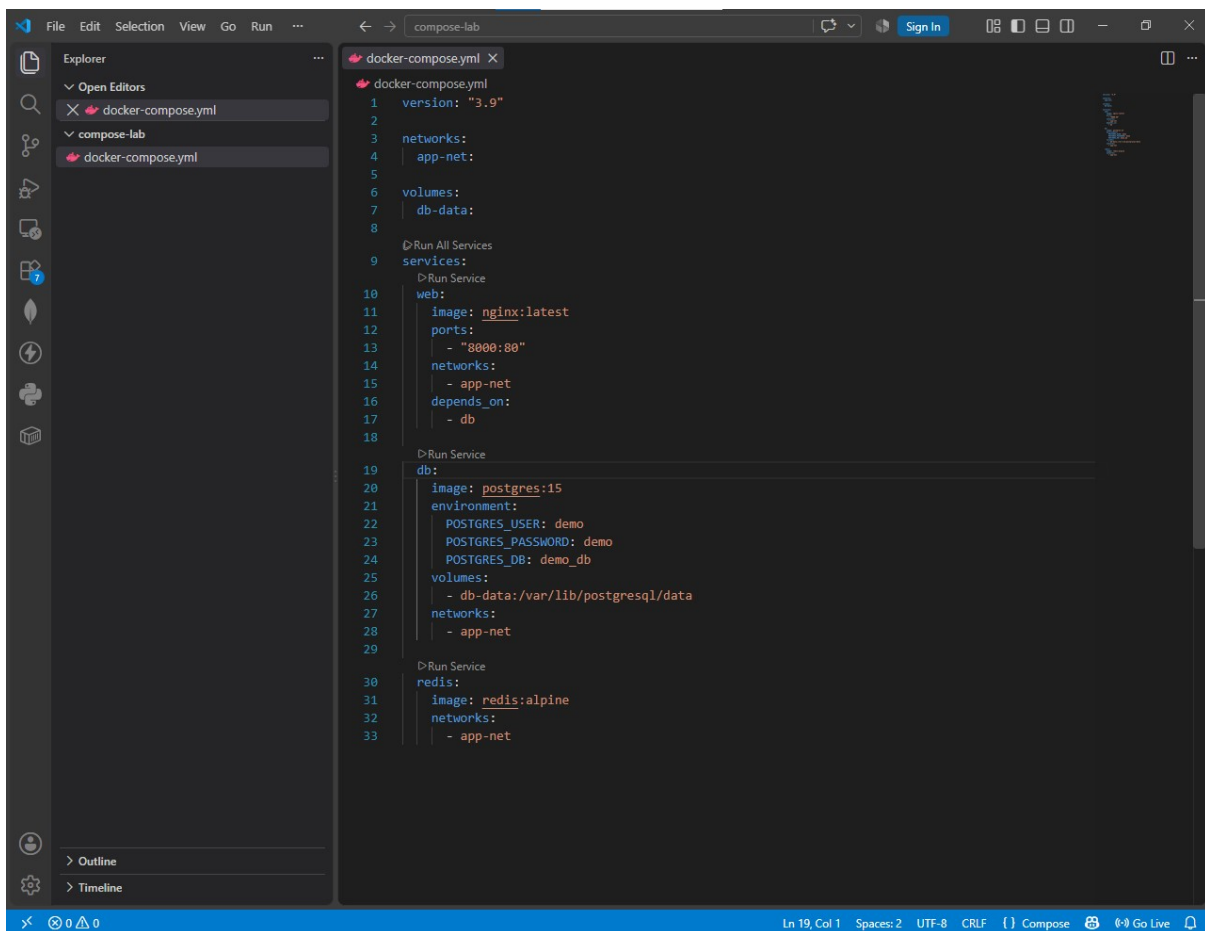
POSTGRES_DB: demo_db

volumes:

- db-data:/var/lib/postgresql/data

networks:

- app-net



II. Run:

docker compose up -d

```

PS C:\Users\fs\Downloads\compose-lab> docker compose up -d
time="2026-09-18T11:07:37+05:30" level=warning msg="C:\Users\fs\Downloads\compose-lab\docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion"
Creating network compose-lab_app-net
Creating volume compose-lab_db-data
Creating container compose-lab-redis-1
Creating container compose-lab-db-1
Creating container compose-lab-web-1
PS C:\Users\fs\Downloads\compose-lab> docker compose ps
time="2026-09-18T11:08:26+05:30" level=warning msg="C:\Users\fs\Downloads\compose-lab\docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion"
NAME                IMAGE                COMMAND               SERVICE    CREATED        STATUS        PORTS
compose-lab-db-1    postgres:15          "docker-entrypoint.s..." db         48 seconds ago Up 43 seconds 5432/tcp
compose-lab-redis-1 redis:alpine          "docker-entrypoint.s..." redis      48 seconds ago Up 43 seconds 6379/tcp
compose-lab-web-1   nginx:latest         "/docker-entrypoint..." web        46 seconds ago Up 42 seconds 0.0.0.0:8000->80/tcp, [::]:8000->80/tcp
  
```

III. Insert some data into Postgres (optional with psql).

IV. Remove containers:

docker compose down

```
PS C:\Users\fs\Downloads\compose-lab> docker compose down
time="2026-09-18T11:10:17+05:30" level=warning msg="C:\\Users\\fs\\Downloads\\compose-lab\\docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion"
Container compose-lab-redis-1 Removed 0.7s
Container compose-lab-web-1 Removed 0.8s
Container compose-lab-db-1 Removed 0.7s
Network compose-lab_app-net Removed 0.5s
PS C:\Users\fs\Downloads\compose-lab> docker ps -a
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS
da84e8e2c8f6   redis:alpine   "docker-entrypoint.s..." 47 hours ago   Exited (255) 31 minutes ago 6379/tcp
243aba36dd18   docer-compos... "python app.py"          47 hours ago   Exited (255) 31 minutes ago 0.0.0.0:5000->5000/tcp
3d3b85ba93ec   postgres:15    "docker-entrypoint.s..." 47 hours ago   Exited (255) 31 minutes ago 5432/tcp
0431a19aaf52   nginx          "/docker-entrypoint..." 2 days ago     Exited (0) 2 days ago
0Fdb9177a80    onlineshoppin... "catalina.sh run"        2 days ago     Exited (255) 31 minutes ago 8080/tcp
```

V. Start again:

docker compose up -d

```
PS C:\Users\fs\Downloads\compose-lab> docker compose up -d
Network compose-lab_app-net Created 0.1s
Container compose-lab-db-1 Started 1.0s
Container compose-lab-redis-1 Started 1.0s
Container compose-lab-web-1 Started 1.0s
PS C:\Users\fs\Downloads\compose-lab> docker compose ps
time="2026-09-18T11:11:06+05:30" level=warning msg="C:\\Users\\fs\\Downloads\\compose-lab\\docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion"
NAME                IMAGE          COMMAND                  SERVICE  CREATED        STATUS        PORTS
compose-lab-db-1    postgres:15    "docker-entrypoint.s..." db        8 seconds ago  Up 6 seconds  5432/tcp
compose-lab-redis-1 redis:alpine    "docker-entrypoint.s..." redis    8 seconds ago  Up 6 seconds  6379/tcp
compose-lab-web-1   nginx:latest   "/docker-entrypoint..." web       8 seconds ago  Up 5 seconds  0.0.0.0:8000->80/tcp, [::]:8000->80/tcp
```

VI. Expected Output:

Database data persists across restarts.

Services communicate via the app-net network using service names.

5.Faster iteration during development

Task:

I. Create a simple Flask app in app.py:

```
from flask import Flask
```

```
app = Flask(__name__)
```

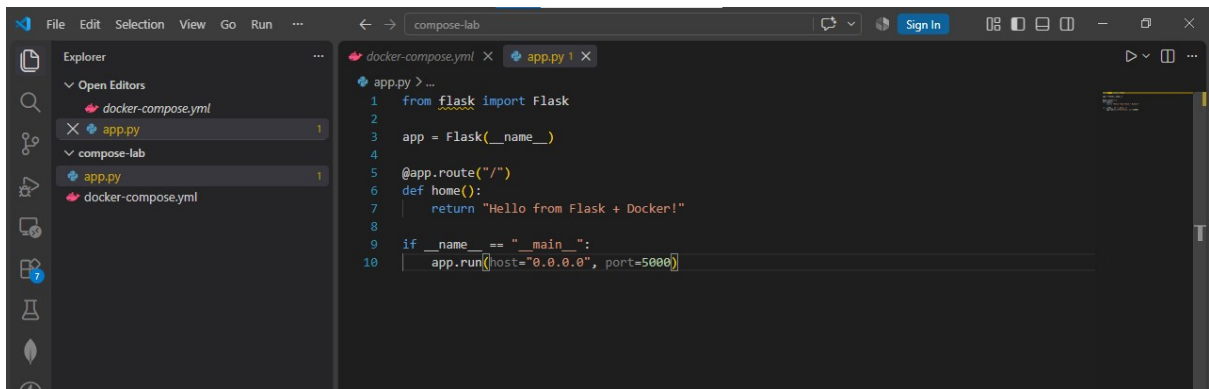
```
@app.route("/")
```

```
def home():
```

```
    return "Hello from Flask + Docker!"
```

```
if __name__ == "__main__":
```

```
    app.run(host="0.0.0.0", port=5000)
```



II. Add a Dockerfile in the same folder:

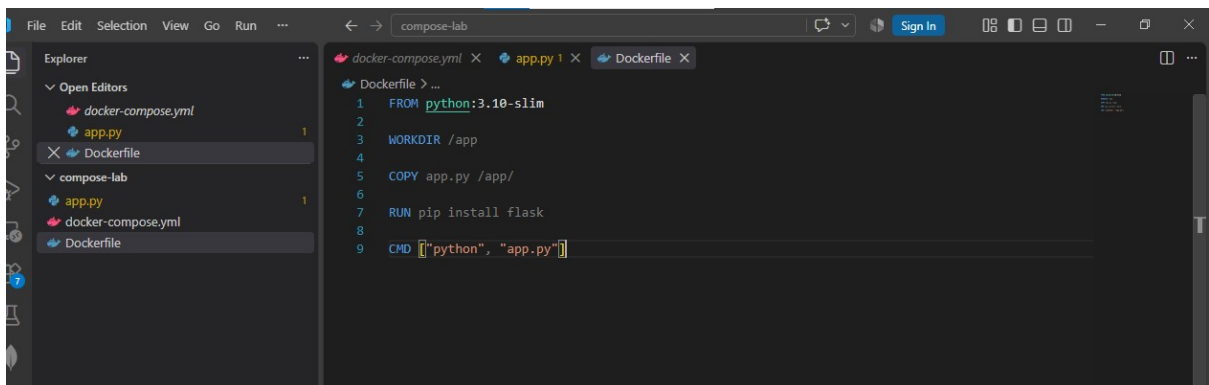
FROM python:3.10-slim

WORKDIR /app

COPY app.py /app/

RUN pip install flask

CMD ["python", "app.py"]



III. Update docker-compose.yml:

web:

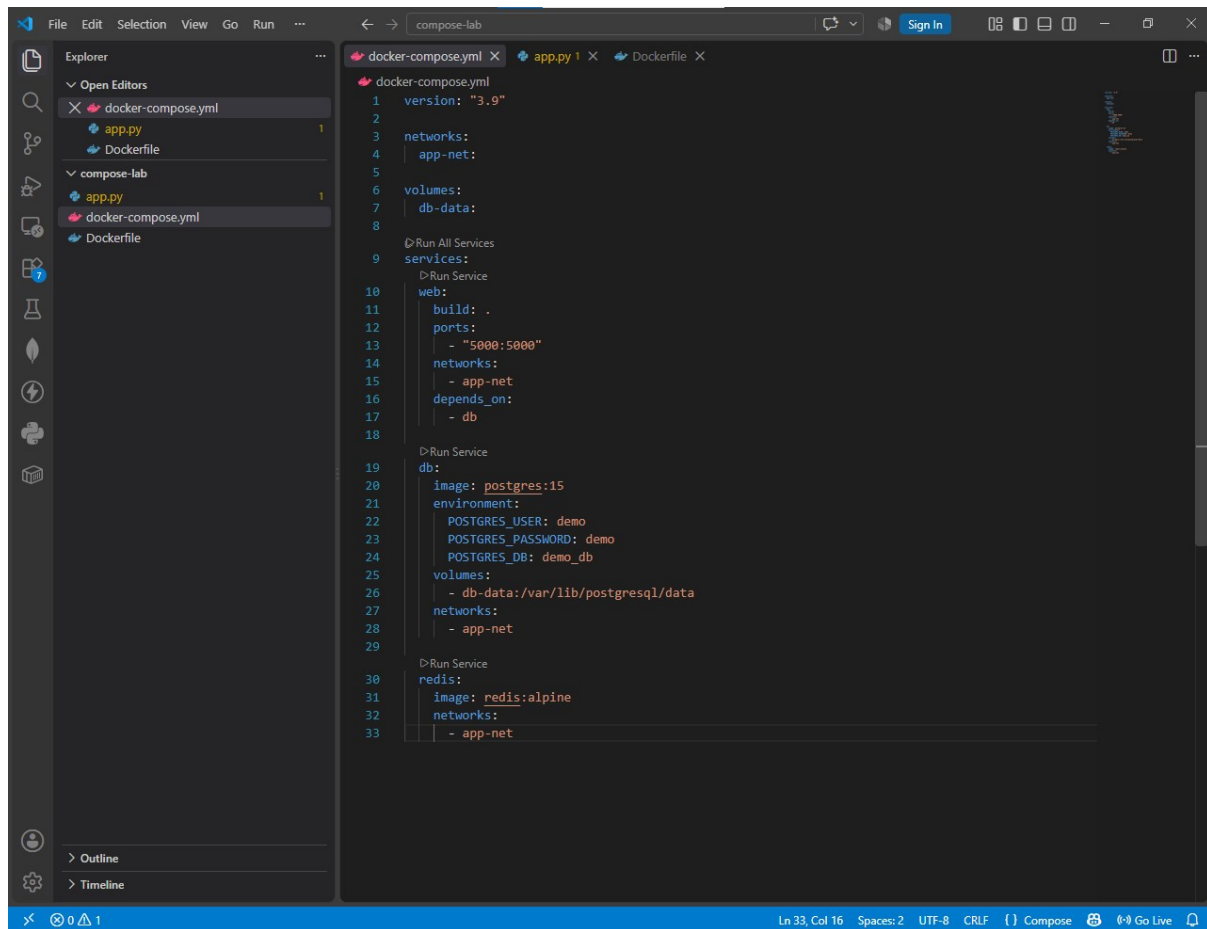
build: .

ports:

- "5000:5000"

depends_on:

- db



IV. Run:

`docker compose up --build`

```

PS C:\Users\fs\Downloads\compose-lab> docker compose ps
time="2026-09-18T11:11:06+05:30" level=warning msg="C:\Users\fs\Downloads\compose-lab\docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion"
NAME                IMAGE                COMMAND                SERVICE    CREATED      STATUS        PORTS
compose-lab-db-1    postgres:15         "docker-entrypoint.s..." db         8 seconds ago Up 6 seconds  5432/tcp
compose-lab-redis-1 redis:alpine         "docker-entrypoint.s..." redis     8 seconds ago Up 6 seconds  6379/tcp
compose-lab-web-1   nginx:latest        "/docker-entrypoint..." web        8 seconds ago Up 5 seconds  0.0.0.0:8000->80/tcp, [::]:8000->80/tcp
[+] Building 4.6s (11/11) FINISHED
=> [internal] load local bake definitions                                0.0s
=> => reading from stdin 524B                                           0.0s
=> [internal] load build definition from Dockerfile                    0.2s
=> => transferring dockerfile: 148B                                     0.0s
=> [internal] load metadata for docker.io/library/python:3.10-slim    2.0s
=> [internal] load .dockerignore                                         0.1s
=> => transferring context: 2B                                           0.0s
=> [1/4] FROM docker.io/library/python:3.10-slim@sha256:fd76ade0c607f27677bc04be3c60749f400e9dc941d9e72967e19a4cedff80c2  0.1s
=> => resolve docker.io/library/python:3.10-slim@sha256:fd76ade0c607f27677bc04be3c60749f400e9dc941d9e72967e19a4cedff80c2  0.1s
=> [internal] load build context                                       0.1s
=> => transferring context: 226B                                         0.0s
=> CACHED [2/4] WORKDIR /app                                           0.0s
=> CACHED [3/4] COPY app.py /app/                                       0.0s
=> CACHED [4/4] RUN pip install flask                                  0.0s
=> => exporting to image                                                0.4s
=> => exporting layers                                                  0.0s
=> => exporting manifest sha256:aa2b08889576cb0d512c64d3b034bec922f877d7faba50efb5f177cc14bac658  0.1s
=> => exporting config sha256:48d39312c21e30ed5c3c3aa0be77f952bf22d16a3986c1c9ebce71eeb6c3a2cb  0.0s
=> => exporting attestation manifest sha256:912312d7ef67c5024b00bc508e43949cb072cc161e2b66307c380b7cdeeda52  0.1s
=> => exporting manifest list sha256:0fa6960872ca123549cd654699d1519989084dc34c1efdb836e654c27e75e3e8  0.1s
=> => naming to docker.io/library/compose-lab-web:latest              0.0s
=> => unpacking to docker.io/library/compose-lab-web:latest           0.0s
=> resolving provenance for metadata file                               0.1s
compose-lab-web      Built
Container compose-lab-redis-1 Running
Container compose-lab-db-1 Running
Container compose-lab-web-1 Recreated
Attaching to db-1, redis-1, web-1
web-1 | * Serving Flask app 'app'
web-1 | * Debug mode: off
web-1 | WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
web-1 | * Running on all addresses (0.0.0.0)
web-1 | * Running on http://127.0.0.1:5000
web-1 | * Running on http://172.22.0.4:5000
web-1 | Press CTRL+C to quit
View in Docker Desktop View Config Enable Watch

```

Visit <http://localhost:5000>.

Change the return text in app.py (e.g., "Hello Docker Compose!").

```

< > ↻ local:5000
Hello from Flask + Docker!

```

V. Rebuild:

docker compose up --build // No need to rebuild

```

Windows PowerShell
Container compose-lab-db-1 Stopped

PS C:\Users\fs\Downloads\compose-lab> docker compose up --build
[+] Building 12.6s (11/11) FINISHED
=> [internal] load local bake definitions
=> => reading from stdin 524B 0.0s
=> [internal] load build definition from Dockerfile 0.0s
=> => transferring dockerfile: 148B 0.0s
=> [internal] load metadata for docker.io/library/python:3.10-slim 1.0s
=> [internal] load .dockerignore 0.0s
=> => transferring context: 2B 0.0s
=> [1/4] FROM docker.io/library/python:3.10-slim@sha256:fd76ade0c607f27677bc04be3c60749f40eedc941d9e72967e19a4cedff80c2 0.1s
=> => resolve docker.io/library/python:3.10-slim@sha256:fd76ade0c607f27677bc04be3c60749f40eedc941d9e72967e19a4cedff80c2 0.1s
=> [internal] load build context 0.0s
=> => transferring context: 220B 0.0s
=> CACHED [2/4] WORKDIR /app 0.0s
=> [3/4] COPY app.py /app/ 0.1s
=> [4/4] RUN pip install flask 7.7s
=> exporting to image 2.5s
=> => exporting layers 1.6s
=> => exporting manifest sha256:5f4fa84c49729367f99b3867862e1d8d615fd6ff173c1f31803f3f0b7047033 0.1s
=> => exporting config sha256:49a2e7826d45144392862e91970e522de26c0900e3be45b59ebb34d277244bc 0.0s
=> => exporting attestation manifest sha256:e9faa0021747e0b23b05c909ffcc37aalf30a378fb3fb9e41b9b3751e447559cf 0.1s
=> => exporting manifest list sha256:d8ab46a3805c0de16dfe764f0db11e27b686729ebc5a882d119515a27126ee8 0.0s
=> => naming to docker.io/library/compose-lab-web:latest 0.0s
=> => unpacking to docker.io/library/compose-lab-web:latest 0.5s
=> => resolving provenance for metadata file 0.0s
[+] compose-lab-web Built 0.0s
[+] Container compose-lab-web-1 Recreated 0.5s
Attaching to db-1, redis-1, web-1
redis-1 | Starting Redis Server
redis-1 | 1:C 18 Sep 2026 05:55:22.024 * oO0oO0oOoOoO Redis is starting oO0oO0oOoOoOo
redis-1 | 1:C 18 Sep 2026 05:55:22.024 * Redis version=8.10.1, bits=64, commit=00000000, modified=1, pid=1, just started
redis-1 | 1:C 18 Sep 2026 05:55:22.024 * Configuration loaded
redis-1 | 1:M 18 Sep 2026 05:55:22.024 * monotonic clock: POSIX clock_gettime

redis-1 | 1:M 18 Sep 2026 05:55:22.027 * Running mode=standalone, port=6379.
db-1 |
redis-1 | 1:M 18 Sep 2026 05:55:22.029 * <bf> RedisBloom version 8.10.1 (Git=unknown)
db-1 | PostgreSQL Database directory appears to contain a database; Skipping initialization

db-1 |
redis-1 | 1:M 18 Sep 2026 05:55:22.029 * <bf> Registering configuration options: [

redis-1 | 1:M 18 Sep 2026 05:55:22.029 * <bf> { bf-error-rate : 0.01 }

db-1 | 2026-09-18 05:55:22.333 UTC [1] LOG: starting PostgreSQL 15.19 (Debian 15.19-1.pgdg13+2) on x86_64-pc-linux-gnu, compiled by gcc (Debian 14.2.0-19) 14.2.0, 64-bit
redis-1 | 1:M 18 Sep 2026 05:55:22.029 * <bf> { bf-initial-size : 100 }

db-1 | 2026-09-18 05:55:22.333 UTC [1] LOG: listening on IPv4 address "0.0.0.0", port 5432
redis-1 | 1:M 18 Sep 2026 05:55:22.029 * <bf> { bf-expansion-factor : 2 }

db-1 | 2026-09-18 05:55:22.334 UTC [1] LOG: listening on IPv6 address ":::", port 5432

redis-1 | 1:M 18 Sep 2026 05:55:22.029 * <bf> { cf-bucket-size : 2 }

```

Visit <http://localhost:5000>.

VI. Expected Output:

New message appears instantly after rebuild.



Scenario based Questions:

1. You have two applications — a Node.js backend and a Python script that handles scheduled jobs. You're considering running both inside the same container. Should you run multiple applications in the same Docker container?

Answer: No. Normally, one main application or service should run in one container. Use separate containers for the Node.js backend and Python script and manage them using Docker Compose.

2. You have a Flask API and an Nginx server. You want to run both containers on the same host and expose them on ports 5000 and 80 respectively. How can you expose both applications from different containers?

Answer:

```
Flask: 5000:5000  
Nginx: 80:80
```

3. You want to run a React frontend, an Express.js backend and MongoDB – all together. How do you run and manage all three together?

Answer: Define React, Express.js and MongoDB as separate services in a `docker-compose.yml` file and run them together using:

```
docker compose up -d
```

4. You try to run two containers that both expose port 8080 on the host. What happens, and how can you run both apps?

Answer: The second container cannot use host port 8080 because it is already occupied. Use different host ports:

```
Container 1: 8080:80  
Container 2: 8081:80
```

5. You updated some code and want to restart your frontend, backend, and DB containers quickly. What is the easiest way to restart all services?

Answer:

```
docker compose restart
```

6. Your frontend has a new release, but you don't want to restart the backend or DB. How can you update and rebuild only the frontend container?

Answer:

```
docker compose up --build -d frontend
```

7. Tomcat is listening on port 8080, but you get a "Connection Refused" error in the browser. Why?

Answer: The container port may not be mapped to the host, Tomcat may not be running properly, or port 8080 may not be accessible. Check the container status and port mapping.

8. You're not sure which container is using port 3000. How do you check?

Answer:

```
docker ps
```

Check the **PORTS** column to see which container is mapped to host port 3000.

9. How do you stop and clean up your running Tomcat container and image?

Answer:

```
docker stop <container>
docker rm <container>
docker rmi <image>
```

10. You want to share your app with a teammate. What do you do with the Docker image?

Answer:

```
docker push username/image
docker pull username/image
```

Conclusion:

Docker Compose simplifies multi-container application management by allowing developers to define, configure, and run services in a single YAML file. It streamlines deployment, ensures consistent environments, and reduces manual setup, making it an essential tool for orchestrating complex, container-based applications efficiently.